



Projecto

Programação Orientada a Objectos, Estruturas de Dados, Manipulação de Ficheiros, Acesso a Bases de Dados Relacionais, Interfaces Gráficas

1. Introdução

Neste projecto, pretende-se a implementação de uma aplicação completa em linguagem Java, com componente gráfica para interacção com o utilizador, recorrendo ao paradigma de Programação Orientada a Objectos e com suporte para bases de dados relacionais.

Espera-se o desenvolvimento e aplicação de conhecimentos e competências relacionadas com as temáticas abordadas na disciplina, com especial incidência para a:

- **programação orientada a objectos**;
- **manipulação de bases de dados relacionais através de JDBC**;
- uso de **ficheiros** para armazenamento de dados de forma persistente;
- bem como o desenvolvimento de **interfaces para interacção com o utilizador** (em modo gráfico).

2. Objectivos

Existe um conjunto de requisitos que o projecto deve respeitar e implementar (os requisitos na secção **Requisitos específicos para a época de exame** apenas se aplicam às Épocas da Avaliação por Exame, não sendo desenvolvidos durante a Avaliação Periódica):

2.1. Descrição global da aplicação

[R1] Completar e corrigir a aplicação iniciada no projecto 1.

Gestão de acesso e utilizadores

[R2] Além dos atributos definidos no projecto 1, os **utilizadores** possuem uma foto, que pode ser inserida ou alterada em qualquer momento.

Caso o utilizador não defina a sua foto, deve existir uma imagem por defeito/genérica no sistema.

[R3] (Para grupos) No momento de registo deve ser enviado um email a informar que o registo foi realizado.

Acções

[R4] Em todos os formulários de inserção de dados deve existir um campo adicional (JTextField ou JAreaField) para inserir observações ou comentários.

- [R5] Após autenticação, deve surgir uma caixa de diálogo ou uma zona na janela principal em realce a indicar a existência de notificações, caso existam.
- [R6] (**Para grupos**) Deve ser possível imprimir um extracto das acções de um processo (indicação das datas e dos utilizadores que realizaram acções num processo).

Listagens e pesquisas

- [R7] Todas as listagens devem incluir barras de deslocamento para permitir visualizar os resultados.

2.2. Gestão geral da aplicação

Interacção com o utilizador

- [R8] Toda a interacção com o utilizador (apresentação de informação ou manipulação de dados) deverá ser realizada através de **interfaces gráficas** (componentes gráficas como janelas, painéis, caixas de diálogos, menus, abas, tabelas, botões, caixas de texto, etc.) disponíveis através da biblioteca gráfica *JavaSwing* / *JavaFX*.

A organização da interface e escolha de componentes fica ao critério do aluno. No entanto, **as interfaces gráficas devem ser intuitivas, limpas e claras**. Soluções que não respeitem estes princípios, independentemente da sua funcionalidade, serão penalizadas.

Soluções eficientes de disposição e transição de componentes, respeitando as regras de interacção com o utilizador serão valorizadas.

- [R9] Devem ser usadas componentes gráficas adequadas à informação a apresentar.

Por exemplo, na apresentação de dados estruturados (listagens) devem ser usadas listas (*JList*) para listagens simples (uma coluna) ou tabelas (*JTable*) para listagens estruturas (várias colunas), e não em texto simples (*JLabel* ou *TextField*) ou painéis (*JPanel*).

- [R10] Os diversos formulários devem possuir uma componente de ajuda.

Sobre cada elemento da interface (e.g. campo de texto, botões) deve existir uma região onde sobrepondo o rato permita o aparecimento de uma nova janela *pop-up* com a informação de ajuda (*tooltips*).

Geral

- [R11] Não é necessário apresentar graficamente mensagens de boas-vindas no arranque e de encerramento, nem estatísticas de utilização, como data, duração ou número de execuções da aplicação.

- [R12] Não é necessário implementar graficamente a manipulação de peças e testes.

- [R13] Não é necessário gestores poderem criar outros gestores através da interface gráfica.

- [R14] O **código deve estar harmonizado**: a mesma funcionalidade (e.g. listagens, gestão de eventos) em diferentes pontos do código deve estar implementada de forma idêntica, sendo consistente, seguindo a mesma abordagem e algoritmo ao longo de todo o código. Não podem existir abordagens distintas ao longo do código para as mesmas funcionalidades, devendo o código ser coerente.

Este requisito torna-se particularmente relevante em projectos desenvolvidos em grupos. Antes do início do desenvolvimento devem acordar nas normas a seguir. Antes da submissão do trabalho devem rever, analisar e harmonizar todo o código para garantir a sua consistência e coerência.

2.3. Requisitos específicos para a época de exame

Comunicação em Rede

- [R15] Deve existir uma **aplicação servidor** (gere todo o processo de gestão de utilizadores e reparações) e uma (ou várias) **aplicação(ões) cliente** (que permita realizar as acções associadas ao utilizador cliente).

A aplicação servidor pode ser acedida por todos os utilizadores.

[R16] A aplicação cliente **não deve armazenar localmente qualquer informação**.

A aplicação cliente **deve comunicar através da rede com a aplicação servidor** para manipular qualquer informação, armazenada na base de dados.

A aplicação cliente só deve ser acedida por clientes.

[R17] A definição do protocolo de comunicação entre cliente e servidor fica ao critério do estudante.

[R18] A aplicação servidor deve informar o utilizador dos pedidos que recebe através da rede e da informação que envia à aplicação cliente

[R19] (**Bonificação**) A aplicação tem de garantir acesso a simultâneo de várias aplicações clientes (ambiente *multithreading*).

Gestão de acesso

[R20] A aprovação das contas pode ser realizada por um de dois métodos:

a) manual, por intervenção do gestor, ou

b) de forma automática, sem intervenção do gestor.

Ambos os métodos devem ser implementados.

[R21] **Aprovação manual:** os **gestores** aprovam os pedidos de registo dos utilizadores.

Todos os pedidos de registo de novos utilizadores devem ser notificados aos **gestores** através da aplicação.

Todos os pedidos devem ser aprovados antes de poderem ser usados para autenticação.

[R22] Caso uma conta seja reprovada pelo gestor ou ainda não tenha sido analisada, quando o **utilizador** tentar usar essas credenciais deve surgir uma mensagem informativa.

[R23] **Aprovação automático:** O sistema permite aprovações automáticas sem intervenção dos gestores.

Neste caso, é enviado um código aleatório por *email* que o utilizador deverá inserir numa caixa de activação da aplicação.

Caso o código seja correcto, a conta será automaticamente activada sem intervenção do gestor.

Geral

[R24] Disponibilizar um módulo de estatísticas onde seja possível verificar a percentagem de pedidos aprovados e rejeitados, os funcionários com menor tempo médio de tratamento de pedidos, os clientes com maior número de serviços, ou o tempo médio para tratamento de serviço.

[R25] Deve ser possível exportar uma listagem dos serviços CSV (*Comma Separated Values*), com a estrutura

<data>, <cliente>, <equipamento>, <estado>

ordenados cronologicamente.

Esta acção só é permitida a **funcionários e gestores**.

[R26] Deve ser possível a listagem anterior.

Esta opção é permitida a **funcionários e gestores**.

[R27] Deve ser possível editar a informação da instituição para constar na impressão, incluindo o nome, morada, contacto telefónico e imagem de logotipo.

3. Implementação

O programa deve ser implementado na linguagem Java. Lembre-se que é uma linguagem orientada a objectos, pelo que deverá ter em conta os seguintes aspectos:

- Cada classe deve gerir internamente os seus dados, pelo que deverá cuidar da protecção das suas variáveis e métodos.
- Cada objecto deverá ser responsável por uma tarefa ou objectivo específico, não lhe devendo ser atribuídas funções indevidas.
- Utilize a *keyword static* apenas quando tal se justifique e não para evitar erros do compilador.
- Recomenda-se que elabore um diagrama com as suas classes e objectos antes de iniciar o projecto, para prever a estrutura do projecto.

Tenha ainda em conta os seguintes pontos que serão importantes na avaliação:

Comente as classes, métodos e variáveis públicas segundo o formato Javadoc. Isto permitir-lhe-á gerar automaticamente uma estrutura de ficheiros HTML, descritivos do seu código, que deve incluir no seu relatório.

- Comente o restante código sempre que a leitura dos algoritmos não seja óbvia.
- Tal como sugerido acima, evite o uso abusivo de *static* e de variáveis e métodos *public*.
- Na escolha de nomes para variáveis, classes e métodos siga as convenções adoptadas na linguagem Java.
- Procure uma interface agradável com o utilizador (na obtenção de dados e disponibilização de informação). As entradas de dados por parte do utilizador deverão ser testadas e protegidas contra erros ou falhas que possam surgir.

4. Logística

Prazos e procedimentos de submissão

- Durante o presente ano lectivo apenas existe **um único projecto a ser desenvolvido durante o período de leccionação**, independentemente da época de avaliação a que o estudante se submete (periódica ou exame).
- O projecto pode ser desenvolvido **individualmente ou em grupo (máximo 2 elementos** - neste segundo caso surgem requisitos adicionais, sendo obrigatório separar as tarefas e responsabilidades de cada elemento) na Avaliação Periódica.

Na Época de Avaliação por Exame, o projecto deve ser desenvolvido **individualmente**

- O projecto tem um peso de 30% (6 valores em 20) na avaliação periódica da unidade curricular.
- O projecto tem como **prazo final de entrega o dia 22 de Junho, até às 23h00** para todas as épocas.
- Serão agendadas **apresentações dos projectos e defesas individuais**,
 - Cada estudante/grupo deve **reservar um bloco após a submissão final do projecto**. Os blocos de defesa não podem ser reservados antes da submissão do trabalho.
 - Nas defesas será solicitado que os estudantes expliquem e alterem o código submetido, bem como implementem novo código presencialmente para novos requisitos.
 - Desrespeito pelas regras, não comparência ou reprovação na defesa implica anulação do trabalho e reprovação à avaliação.
- Submissão implica a entrega, em formato digital, das seguintes componentes:
 - **código da aplicação** (ficheiros com extensão `.java`, agrupados num ficheiro `.rar` ou `.zip`, incluindo ficheiros adicionais relevantes);
 - **executável** (ficheiro JAR) que permita a correcta execução da aplicação;
 - **descrição das classes**, métodos e atributos através de um **JavaDoc**;
 - **relatório final** onde seja documentada a aplicação, descrevendo-se as classes manipuladas, bem como as estruturas de dados, algoritmos ou soluções de implementação seleccionadas e respectivas justificações, com **manual de utilizador**.
- Todo o material deve ser agrupado num ficheiro `.rar` ou `.zip`.
- O ficheiro do projecto submetido deve respeitar a nomenclatura seguinte (no caso de trabalhos em grupo devem incluir os nomes de ambos os elementos):

LEI_PA_2026_AP3_Nome-Apelido_2026-06-xx.rar (Avaliação Periódica), ou

LEI_PA_20256_AE_Nome-Apelido_2026-06-xx.rar (Todas as épocas da Avaliação Exame).
- A entrega implica a **submissão exclusivamente através da plataforma Nonio**. Nenhum trabalho será aceite fora desta plataforma ou após terminar o prazo definido (a plataforma de submissão encerrará automaticamente).
- O desenvolvimento ocorrerá tanto em períodos de aulas como extra-aulas.
- Sendo o foco do projecto avaliar as competências em programação do estudante, **todo o código deve ser desenvolvido exclusivamente pelo estudante**. Não são permitidas ferramentas de geração automática de código e/ou motores de Inteligência Artificial (e.g. *CoPilot*, *ChatGPT*, *IDE Code Completion*), nem código produzido por outros elementos. Projectos que incluam código ou indícios de código não desenvolvido pelo estudante serão considerados fraude.
- Situações de **fraude (cópia/plágio)** implicam a **imediata anulação da avaliação (zero valores) e sancionados de acordo com o Estatuto Disciplinar do Estudante**. A cópia de programas resultará na atribuição de nota zero a todos os alunos envolvidos. Também não é permitido copiar código da internet não referenciado.
- Todas as **fontes externas** (e.g. livros, páginas de internet, recursos didáticos, ferramentas) devem ser explicitamente **referenciadas**, seguindo o formato indicado no modelo de relatório.
- Os estudantes devem ter em consideração que apenas serão aceites para avaliação projectos que:

- incluam **todos os elementos de avaliação obrigatórios**,
- possuam **pelo menos 50% dos requisitos totalmente implementados**,
- na avaliação por Exame **todos os requisitos** indicados com a etiqueta **Exame** têm de estar **implementados**,
- a aplicação **execute integralmente** sem anomalias,
- a aplicação servidora seja capaz de ler e escrever numa base de dados relacional,
- a aplicação possua uma interface gráfica funcional, e
- a **totalidade do código é da autoria exclusiva do estudante**, sendo **totalmente proibido incluir código produzido por ferramentas de geração automática de código** (e.g. *CoPilot*, *ChatGPT*, *IDE Code Completion*).

O não cumprimento de um dos requisitos assinalados **implica a anulação do trabalho e consequentemente da avaliação**.

Documentação anexa

- O relatório final deve seguir obrigatoriamente o **modelo disponibilizado**:
 - **Todas as secções do modelo são obrigatórias** e devem ser adequadamente preenchidas. O modelo deve assim ser respeitando na totalidade.
 - **Não podem ser eliminadas do documento secções** e toda a imagem tem de ser acompanhada de legenda e de um texto a descrever e explicar essa imagem.
 - Secções removidas, sem conteúdo, apenas com imagens, ou com texto genérico implicam a anulação do documento.
 - O documento deve possuir um conjunto de **referências obrigatórias**. Todo o material e fontes de apoio e consulta para a elaboração do projecto devem ser incluídas. A não inclusão desta secção implica a anulação do documento.
 - Tal como no desenvolvimento do código do projecto, o relatório deve ser elaborado exclusivamente pelo estudante, **não sendo permitido o uso de ferramentas de geração de texto**, nomeadamente ferramentas de IA.
 - A anulação do relatório, sendo um elemento de submissão obrigatória, **implica a reprovação à avaliação** no presente ano letivo (não é possível nova submissão de projecto).

O não cumprimento de um dos requisitos assinalados **implica a anulação do relatório e consequentemente do relatório**.

- O **relatório deve conter, entre outros, os seguintes tópicos** (não de forma exclusiva, sendo recomendável a inclusão de tópicos adicionais sempre que estes permitam uma melhor clarificação do projecto e tornem o documento mais completo):
 - Descrição dos objectivos a atingir com o projecto;
 - **Indicação explícita dos requisitos implementados e não implementados**, justificando os requisitos incompletos (recomenda-se o uso de uma tabela listando todos os requisitos e o seu estado);
 - **Distribuição das tarefas ao longo do tempo** - indicação do planeamento das tarefas e recursos utilizados (com especial foco no tempo);
 - No caso de trabalhos em grupo deve ser claro **como foi distribuído o trabalho entre os elementos do projecto**, que **responsabilidades cada elemento assumiu**;
 - **Descrição das classes e respectivos métodos** (é aconselhável disponibilizar um **diagrama de classes**, e.g. *VisualParadigm*, *UMLet*, *DIA Diagram Editor*, *Microsoft Visio*);
 - Descrição do **modelo de bases de dados** gerado (diagramas conceptual e físico, e.g. *Sybase/SAP PowerDesigner*, *MySQL Workbench*);
 - Apresentação dos **algoritmos** adoptados;
 - Indicação das **estruturas de dados** implementadas, justificando as escolhas realizadas;

- **Testes** realizados para averiguar a completude dos requisitos e o correcto funcionamento da aplicação;
- Avaliação crítica do projecto: **forças e limitações**;
- Possíveis **melhorias futuras**;
- **Referências bibliográficas** (material e fontes consultadas).
- Em anexo:
 - Manual do utilizador (descrição do funcionamento da aplicação indicando os comandos disponíveis, como executá-los e obtenção de resultados);
 - JavaDoc;
 - Documentação adicional considerada relevante para o projecto.

Parâmetros de avaliação

- O código e execução da aplicação apresenta um peso de 70%, enquanto a documentação (relatório, manual e *JavaDoc*) apresenta um peso de 30%.
- A avaliação dos trabalhos terá em conta vários parâmetros, nomeadamente:
 - Programação Orientada a Objectos;
 - Usabilidade da interacção com o utilizador;
 - Manipulação de ficheiros e de estruturas de dados relacionais;
 - Completude dos requisitos;
 - Funcionalidades disponíveis;
 - Protecções implementadas, tanto na leitura de dados do utilizador, como no acesso a ficheiros;
 - Interacção com o utilizador;
 - Execução da aplicação sem erros;
 - Respeito pelas normas de codificação e documentação;
 - Documentação do código;
 - Pesquisa e aplicação de novos conhecimentos;
 - Domínio da linguagem java;
 - Qualidade do relatório;
 - Qualidade Manual do utilizador e *JavaDoc*;
 - Apresentação e defesa do trabalho realizado;
 - Capacidade de explicar e alterar o código submetido.